VERSION CHANGE REPORT

Omni-Queue 360: old version vs. current version

A detailed comparison across three areas — the AI wait-time engine, the queue logic, and security. Each item states what the old version did, what the current version does, and which real problem it solves. All figures reflect the repository as it stands now.

Old E:\AI Ticket Queue\source-code-main **Current** E:\Final Ai queue\source-code-main

SOURCE LINES	API ENDPOINTS	BACKEND TESTS
6,708 → **11,481**	21 → **33**	0 → **58**
AI ENGINE TESTS	**DATABASE TABLES**	**OPERATING SETTINGS**
9 → **14**	4 → **6**	3 → **21**

PART ONE

AI — the wait-time engine

This is the deepest change. The old version called this module an "AI engine" but contained no machine-learning model at all. The current version has a real model that learns from served tickets, with a gate that blocks a bad model from ever being used. The engine is scoped to one job only: predicting wait time.

The old version was really one division

The old engine exposed only three routes — `/predict`, `/predict_batch`, `/health` — with no training, no stored model, and no ML dependency. The entire "AI" lived in a single function:

```
def calculate_eta(queue_position, average_service_time_mins,
active_staff):
    if queue_position == 0:
        return {"estimated_wait_time_mins": 0, "queue_status":
"Low", "available": True}
    if active_staff <= 0:
        return {"estimated_wait_time_mins": None,
"queue_status": "Unavailable", ...}
    eta = (queue_position * average_service_time_mins) /
active_staff
    return {...}
```

Beyond having no model, this had three behavioural bugs worth naming:

- **Wrong check order.** `queue_position == 0` was tested *before* `active_staff <= 0`. So a customer first in line at a counter with nobody on duty got "0 min / Low / available" — the system promised immediate service for a ticket that would never be called.
- **The backend invented staff.** In the old `aiService.js`, `getActiveStaffCount` ended with `return count > 0 ? count : 1`. With nobody signed in it returned 1, so even the `active_staff <= 0` branch almost never ran — an empty counter still quoted a confident wait.
- **Flat fallback.** When the engine did not answer, or when `AI_ENGINE_URL` was empty (the old default), the backend returned a hard-coded 5 min / "Medium" for *every* position. The first and the twentieth person in line saw the same number.

OLD VERSION

- Fixed `300ms` call timeout — too short for a cold Flask worker on Windows, so the first ticket of every session fell back
- Sent: position, average service time, staff count, service name
- No hour-of-day, no weekday
- No `source` field — could not tell a model estimate from a fallback
- No training loop, no saved model, no history

CURRENT VERSION

- Configurable timeout, default `2000ms`
- Also sends `hour` and `weekday` — the model learns peak-hour rhythm
- `source` returns `"model"` / `"analytic"` / `"fallback"`, shown on the admin screen
- `available: false` when no counter is open, distinct from "0-minute wait"
- Training loop runs in the background every 15 min; model persisted to disk with `joblib`

How the current version works

The engine (`apps/ai-engine/predictor.py`) is three layers stacked. The bottom always answers; the top is used only when it proves better.

LAYER 1 — THE ANALYTIC BASELINE

Still `(position × average service time) ÷ staff`, but the check order is fixed: staff is tested *first*. Nobody on duty → returns `None`, which the whole chain above translates to `queueStatus: "Unavailable"`. This answers every request before any history exists, and is the safety net whenever the model is unavailable.

LAYER 2 — THE LEARNED MODEL

A scikit-learn `GradientBoostingRegressor` (200 trees, learning rate 0.05, depth 3) fitted on tickets that have been called. Each training row has 8 features plus a one-hot per service:

FEATURE	MEANING
`queue_position`	People ahead, recorded at ticket issue
`active_staff`	Staff on duty at ticket issue
`average_service_time_mins`	That counter's average service time at issue
`baseline`	**The analytic estimate itself.** Feeding it in means the model only has to learn this centre's *deviation* from textbook queueing — a far easier target on a few hundred tickets than the raw wait
`position / staff`	Load per counter
`sin, cos (hour)`	Hour of day encoded cyclically — 23:00 sits next to 00:00, not 23 units away
`weekday`	Day of week
`one-hot service`	Each counter has its own rhythm

The label (`actual_wait_mins`) is the wait that *actually happened*: from the moment the ticket entered the queue (`effectiveQueueTime`) to the moment it was called (`calledAt`). That is exactly the quantity the ETA claims to predict.

LAYER 3 — THE ADOPTION GATE

This is the most important part technically. After fitting, the new model is measured on a held-out 25% and compared against the analytic baseline on the same data:

```
if model_mae >= baseline_mae:
    self._services = previous_services   # roll back
    return {"trained": False, "reason": "model did not beat the
analytic baseline", ...}
```

A model that does not beat the formula is discarded and the formula stays in use. **One bad training run can never make estimates worse.** The old version had no such concept, because it had nothing to evaluate.

THE FALLBACK LADDER

- Engine returns a `predictions` array → used directly.
- Engine returns a bare number array (older build) → still normalised.
- Engine down / timeout / garbage reply → falls back to `analyticPrediction()` , i.e. *that same formula* running in Node. Not a flat 5-minute constant.
- On-disk model corrupt or sklearn version-mismatched → caught, model dropped, formula continues.
- Predictions clamped to `[0, 240]` minutes — a model is an estimator, not an oracle.

> THE KEY POINT
>
> Ticket issuance is **never** blocked by the AI. Every error path has an exit back to the formula, all inside `try/catch` . The old version also had try/catch, but its exit was a meaningless constant.

How the AI touches the queue

This is often misunderstood, so it is worth stating plainly: **the AI does not decide serving order.** Order is decided by `effectiveQueueTime` (see Part Two). The AI only produces the displayed number and the status label. It touches the queue in exactly four places:

01	02	03	04
At issue	**Context capture**	**On every status change**	**On completion**
Counts people currently `Waiting` in that service, asks the engine, writes `predictedWaitTime` and `queueStatus` onto the ticket.	`captureModelContext` records position, staff count and average service time *at that moment* onto the ticket.	`updateAllETAs()` sends the whole row `[0,1,2…n]` to `/predict_batch` and rewrites the ETA for the entire line.	The gap between `effectiveQueueTime` and `calledAt` becomes the label of a new training row.

Step 02 is easy to overlook but decides model quality. If we only read "how many staff were on" at training time, we would read the *present* value, not the past one — the model would learn from information the system never had at prediction time. So the context is frozen onto the ticket itself, via four new columns on the `leads` table: `queue_position_at_creation`, `active_staff_at_creation`, `average_service_time_at_creation`, `queue_status`.

Step 03 is also where the old version failed most visibly. Both versions send the *position index* (`0, 1, 2…`), not a single total, so each person should get a different ETA. But because the old fallback returned one value for every element, whenever the engine was not running — which by default it was not, since `AI_ENGINE_URL` was empty — the whole line showed "5 min".

USER IMPACT – OLD	USER IMPACT – CURRENT
Counter with nobody on duty: the kiosk showed "0 min", and a customer waited for a ticket that would never be called.	Counter with nobody on duty: the kiosk shows a "no counter open yet" warning, and the registration form says so before the customer taps to take a ticket.
Engine not running: all 20 people in line saw the same "5 min / Medium".	Engine not running: each person still sees their own ETA by position, just less accurate.
Numbers never improved over time — nothing to learn from.	The more tickets served, the closer the estimate — as long as the model clears the adoption gate.

> SCOPE NOTE
>
> The AI engine is deliberately limited to wait-time prediction. Its five routes — `/predict`, `/predict_batch`, `/train`, `/model`, `/health` — are all prediction-related. It runs no other services and adds no other agents.

PART TWO

Queue logic

The old version had three overlapping ordering rules and no business rules at all. The current version reduces ordering to a single sort key and adds a full business layer between the controller and the store.

Serving order — the most fundamental change

OLD — THREE OVERLAPPING RULES

```
ORDER BY priority DESC, created_at ASC
```

- A boolean `priority` flag
- `source` (On-site / Remote)
- `timestamp`, *overwritten* on every check-in

These interacted, so every combination became its own test case.

CURRENT — ONE SORT KEY

```
ORDER BY COALESCE(effective_queue_time, created_at) ASC, id ASC
```

```
effectiveQueueTime =
  booked ticket → max(appointment, arrival)
  walk-in       → arrival
```

Ties broken by `id` so order never flickers between two reads.

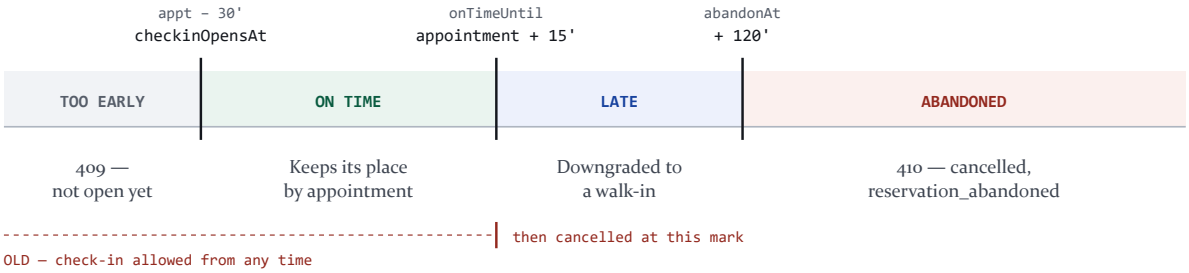
One formula replaces three rules, and it reads out as three clear policies:

- **Arriving early gains nothing.** A 14:00 appointment reached at 13:35 is still ordered by 14:00.
- **Arriving late loses the appointment.** Past the grace period, the sort key becomes the actual arrival time.
- **Walk-ins are never pushed back indefinitely.** A booked ticket no longer has an automatic right to cut in.

> PROBLEM SOLVED
>
> In the old version, `performCheckin` set `lead.timestamp = now` on check-in. Combined with `createdAt = scheduledFor − 5 min`, a booked ticket queued *exactly like* a walk-in that had just stepped in. Booking ahead gave no benefit at all — the feature existed but was meaningless.

Booked-ticket lifecycle and the check-in window

The old version had only one timestamp: `pendingExpiresAt`. Before it, check-in was allowed; after it, the ticket was cancelled outright. There was no lower bound, meaning **a 14:00 appointment could check in at 08:00 and occupy the live queue for six hours.** The current version has three boundaries:



All three markers come from the settings table, not hard-coded: `checkinEarliestMinutes` (30), `checkinGraceMinutes` (15), `lateDowngradeWindowMinutes` (120).

The current logic handles each case as follows:

CASE	WHAT THE SYSTEM DOES	WHAT THE OLD VERSION DID
Too early	Refuses with HTTP 409, states the check-in open time. Audits `too_early`.	Allowed check-in; ticket entered the live queue immediately
On time	`effectiveQueueTime = max(appointment, now)`, keeps its place by appointment	`timestamp = now` — lost its appointment place
Late, within 2 h	`walkInDowngraded = true`, ordered by arrival. **Ticket not cancelled.** Someone who has already travelled here is still served.	Cancelled; the customer had to book again
Past 2 h	Cancelled with reason `reservation_abandoned`, HTTP 410	Already cancelled long before
Check-in pressed twice	Reports success, returns `alreadyCheckedIn: true` — idempotent	Showed a red error to someone who had just checked in
Wrong email/phone	Counts the failure; 5 failures → 15-minute lockout, HTTP 429	Logged but never counted — unlimited guessing

Business checks before a ticket is created

This whole layer (`businessService.js`) is new. The old version checked nothing: the service name was free text, hours were not enforced, and slot capacity did not exist.

CHECK	CURRENT VERSION	OLD VERSION
Service	`requireService()` — must exist in the catalogue and be active (404 / 409)	Free text. One typo created a queue nobody serves

CHECK	CURRENT VERSION	OLD VERSION
Past appointment	Refused, asks for a later time	*Silently* moved to now — the customer thought they held 9:00 while the system held the present moment
Opening hours	Business days + holidays + open window. A 3:00 am Sunday booking is blocked	Not checked
Booking horizon	`bookingHorizonDays` = 7 days	None
Slot capacity	`slotCapacity ?? counters × slotCapacityPerCounter`, counted by `countBookingsInSlot`	`MAX_ONLINE_ETA_MINUTES` — measured the *current* queue, which says nothing about a slot three days out
Walk-in cut-off	Stops issuing 30 min before close, so everyone already waiting is served	None
Per-customer cap	1 live ticket per service, at most 2 concurrent services	None — one person (or one script) could hold every slot

A concurrency detail worth noting: slot capacity is checked **twice** — once when the request is validated, and again *inside the lock* before the write. Two people can both pass validation for the last seat, but only one gets the write.

Staff actions at the counter

ACTION	CURRENT LOGIC	OLD VERSION
call-next	Locks `queue:position:<service>`, picks the ticket with the earliest `effectiveQueueTime`, sets `Called` + `calledAt` + `recallCount = 0`. A counter can only call its own line unless it explicitly sends `coveringFor: true`; admins may call any line.	No assignment check at all — counter A silently called counter B's customers
recall	Up to `maxRecalls` = 2. The third recall auto-converts to `No-Show` with reason `recall_limit_reached`, returning `autoNoShow: true` so the counter knows what actually happened.	`recallCount` incremented forever, read by nobody. One ticket could block a counter indefinitely
no-show	Only from `Called`, writes `cancelReason: marked_by_staff`	Only from `Called`, no reason recorded
transfer	Only from `Called` or `Serving` (409 otherwise). Target must exist in the catalogue and differ from the current one. **`effectiveQueueTime` is deliberately kept** — the customer keeps the wait already served and does not jump to the front of the new line.	Worked from *any* status including `Completed` and `Cancelled`. Set `priority = true` and `timestamp = now` → erased the completion timestamp and pushed a finished ticket to the front of a live line
cancel	New endpoint. A customer can self-cancel until called (`Pending` / `Waiting`); staff can cancel on their behalf in any active state. Ownership is checked.	Did not exist. A customer who wanted to give up their place could only wait for the ticket to expire

Background maintenance — the self-cleaning queue

The old version had `expirePendingLeads`, run every 30 seconds, doing one thing: cancelling expired `Pending` tickets. **Every other state could only be left by a manual staff action.** A staff member who closed their browser tab left a ticket stuck in `Called` or `Serving` forever: it blocked the counter, never reached the statistics, and nothing in the system would ever move it.

The current version renames this to `runQueueMaintenance`, still every 30 seconds, handling four situations, each ticket wrapped in its own `try/catch` so one unwritable ticket cannot stall the sweep:

01	02	03	04
Stale day-old ticket	**Abandoned booking**	**Called, no-show**	**Long session**
Live ticket belonging to a previous day → cancelled, reason `not_served_before_closing`. Toggle off with `carryOverWaitingTickets`.	`Pending` past `pendingExpiresAt + 120'` → cancelled, reason `reservation_abandoned`.	`Called` past 5 min → auto `No-Show`, reason `auto_no_show_timeout`. Counter freed.	`Serving` past 45 min → emits `long_session_alert` to admins with the elapsed minutes. **Never auto-closes** — it only prompts a person.

Marker 04 is a deliberate policy choice: a 50-minute consultation is normal, so the system alerts a manager rather than ending it. `longSessionAlertedAt` ensures the alert fires once per session. If any ticket is closed during a sweep, the whole line's ETA is recalculated immediately.

Operating rules become data

OLD — ENVIRONMENT VARIABLES

Three parameters, in `config/index.js`:

- `ONLINE_EARLY_CREATE_MINUTES`
- `ONLINE_CHECKIN_GRACE_MINUTES`
- `MAX_ONLINE_ETA_MINUTES`

Changing policy meant editing `.env` and restarting the server.

CURRENT — THE `APP_SETTINGS` TABLE

21 parameters in `DEFAULT_BUSINESS_SETTINGS`, seeded once, then owned by the admin *Operations* screen.

Opening hours, holidays, slot length, capacity, booking horizon, the timeouts, recall cap, per-customer caps, whether cross-counter calls are allowed, whether the kiosk phone is mandatory…

Read with a 5-second cache, invalidated on every write. Changing policy never means touching code.

Service catalogue

In the old version the service list was hard-coded across **four front-end files** — `LanguageContext.tsx` (kiosk), `OnlinePlatform.tsx`, `StaffWorkspace.tsx`, `StaffManagementView.tsx` — while the backend validated nothing. Renaming a service meant editing four places, and any drift created tickets in a queue nobody could see.

The current version has a `services` table, a public `GET /api/catalog` endpoint, and a shared `useCatalog` hook. One notable detail: when an admin deletes a service that still has live tickets, the system *does not delete* it — it deactivates it with an explanatory message, because a real delete would strand those tickets in a queue that no longer exists.

Runs anywhere

OLD VERSION

Redis was mandatory. `initSocket` always dialled Redis to attach the adapter, so with no Redis the server could not boot.

Worse: with a `reconnectStrategy` that always returned a delay, `connect()` against a port nobody listened on *never settled*. The server hung, printing "Redis reconnecting..." until killed — no error, no exit.

CURRENT VERSION

`memoryClient.js` — an in-process stand-in implementing exactly the command surface the rest of the code uses, with the same return shapes as node-redis v5.

`QUEUE_STORE=auto` (dev default) probes for 1.2 s then falls back to in-process; `redis` (production default) requires it; `memory` never touches Redis.

`connectWithDeadline` puts a hard deadline on `connect()`, turning an infinite hang into an error the caller can handle.

Real-time communication

PROBLEM SOLVED

The old version used `getIo().emit(...)` in `saveAndBroadcastLead`, `createLead`, `cancelExpiredLead`, `bookTicket` and `submitFeedback` — i.e. a **global broadcast to every connected socket**. Every customer's browser received every update of every ticket. Rooms existed in the code but were bypassed by the main flow.

The current version addresses events to exactly three kinds of room, with no global broadcast:

- `admin_room` — the management dashboard
- `position_<service>` — the counters on that line; staff join it automatically from their token
- `ticket_<number>` — the one customer holding that ticket; the customer token carries the ticket number so the socket joins its own room

`broadcastLead()` sends to exactly those three targets. The admin dashboard is event-driven with a 30-second safety poll (10 s when disconnected), instead of polling every endpoint every 5 seconds as the old version did.

PART THREE

Security

The old version's foundation was already reasonable — helmet, a CORS allow-list, JWT, per-route RBAC, timing-safe comparison, PII masking in logs, and a correctly-written distributed lock. The changes below close application-layer gaps: per-resource authorization, anti-enumeration, and data integrity.

Vulnerabilities closed

AREA	OLD VERSION	CURRENT VERSION
Socket leakage	Global broadcast — every customer received every ticket's updates. Data was reduced to public fields by `publicLead` so no PII leaked, but the entire queue flow still reached every browser	Room-addressed. A customer token can only reach its own `ticket_<number>` room
Counter authorization	**None.** Any staff token could call / recall / no-show a ticket in *any* line. The service came from the browser request and was never compared against the signed-in staff member's assignment	`assertMayServe` / `assertMayActOnLead`. Covering another line requires an explicit `coveringFor`; every action taken while covering is logged and pushed to admins via `staff_cross_counter`
Check-in enumeration	Failures were audited but **not counted**. Another customer's email/phone could be guessed without limit	Counted per `ticket : hash(IP)` scope. 5 failures → 15-minute lockout, HTTP 429. Success clears the counter. The TTL refreshes on each failure so a slow attack cannot slip through the window
Ticket lookup	Endpoint did not exist — losing the number meant losing the place	Adds `POST /api/leads/lookup`, but because it is an enumeration oracle by nature it has its own limiter: `30` requests / 15 min, versus `1000` for the other public endpoints
Booking verification	None. One POST created a real ticket	6-digit OTP, stored as SHA-256, 10-minute TTL, max 5 attempts, single-use, purpose-bound. **The validated booking travels inside the challenge**, so details cannot be swapped between requesting and redeeming the code
Feedback integrity	Ratable in *any* state, overwritable without limit. The dashboard CSAT figure was unauditable	`Completed` tickets only, once, with an ownership check (409 on violation)
Token lifetime	One `JWT_EXPIRES_IN` = 12h shared by admin, staff and customer	Split: staff/admin sessions expire with the shift (12h), a customer's ticket token lasts 7 days — because the ticket outlives a shift
Service validation	Free text everywhere. You could even create a staff account assigned to a nonexistent service — that person would never receive a customer	`requireService()` on every entry point, including creating and editing staff accounts
Lock in tests	`if (config.isTest) return routine({ aborted: false })` — **the lock was skipped entirely under test**. The locking path was never exercised once	Removed. Tests run through the real lock on the in-process backend

AREA	OLD VERSION	CURRENT VERSION
Lock scope	`locks:queue:global` even for single-ticket actions — a global lock for a local operation	Per-resource: `queue:lead:<id>`, `queue:ticket:<number>`, `queue:position:<service>`. `queue:global` is used only when issuing a ticket
OTP code in logs	—	`OTP_DEV_ECHO` is hard-disabled when `NODE_ENV=production`, regardless of the env var
Audit trail	`checkin_audit` recorded basic reasons	Adds `locked_out`, `too_early`, `reservation_abandoned`; plus a separate log for every cross-counter action

What was already good and was kept

- `safeCompare` hashes both sides with SHA-256 before `timingSafeEqual` — both timing-safe and free of buffer-length errors. Unchanged.
- The logger recursively masks `phone` and `email` in every metadata object. Unchanged.
- The distributed lock uses a per-holder token, released and extended by a Lua script that compares the token first — a slow request cannot delete a lock another has since acquired. If the lock backend is unreachable the request *fails* rather than proceeding unlocked. Unchanged (only the in-process branch was added).
- Production guards that require `JWT_SECRET`, `ADMIN_USERNAME`, `ADMIN_PASSWORD`, `STAFF_PIN`. Unchanged.
- Helmet, an explicit CORS allow-list in production, a 1 MB body limit. Unchanged.

> NOTE FOR THE DEFENCE
>
> The strongest point to present here is not the count of vulnerabilities closed, but the **per-resource authorization gap**: the old version authenticated correctly (it knew who you were) but did not authorize (it never checked what you were allowed to act on). That is *Broken Access Control* — number one in the OWASP Top 10. The `assertMayServe` / `assertMayActOnLead` pair is the direct answer.

In summary

The three parts share one motif. The old version had the *shape* of features but was missing the core: an "AI engine" with no model, a booking system where booking ahead gave no benefit, an authentication layer with no authorization, and socket "rooms" the main flow bypassed.

The current version replaces each empty part with a real mechanism, and — just as important — adds a way to prove those mechanisms work: `58` backend tests (the old version had `0`, despite Jest and Supertest being installed) plus `14` AI-engine tests. The AI itself is now scoped to a single job: predicting wait time, and nothing else.